

Github Project: Git Commands Documentation Template

You will use this template to copy and paste the git commands you used to complete all tasks on your local and remote git repository for this project. This file will serve as your submission for the GitHub project.

Instructions:

1. Make a copy of this Git Commands Documentation template on your Google Drive.
2. Complete the four sections in this document with the appropriate git commands.
3. Download this document as a PDF file.
4. Submit this on the Project Submission page within the Udacity Classroom.

1. Set Up Your Repository

The following are the steps you will take to create your git repository, add your code, and post your files on GitHub.

Step 1. Create a GitHub profile (if you don't already have one).

Step 2. Fork a repository from Udacity's [GitHub Project repository](https://github.com/Summer-Yu-1121/post-your-work-project) and provide a link to your forked GitHub repository here:

GitHub Repository Link
https://github.com/Summer-Yu-1121/post-your-work-project

Step 3. Complete the tasks outlined in the table below and copy and paste your git commands into the "Git Commands" column. The first git command is partially filled out for you.

	Tasks	Git Commands
A.	Clone the GitHub repository to your local repository.	git clone https://github.com/Summer-Yu-1121/post-your-work-project.git
B.	Move your bikeshare code and data files into your local repository.	No git command needed
C.	Create a .gitignore file containing the name of your data file.	bash touch .gitignore
D.	List the file names associated with the data files you added to your .gitignore	No git command needed
E.	Check the status of your files to make sure your files are not being tracked	bash git status
F.	Stage your changes.	bash git add .
G.	Commit your changes with a descriptive message.	Commit your changes with a descriptive message. bash git commit -m "chore: Initial setup with bikeshare script and gitignore"
H.	Push your commit to your remote repository.	Push your commit to your remote repository. bash git push origin main

2. Improve Documentation

Now you will be working in your local repository, on the BikeShare code file and the README.md file. You should repeat steps **C** through **E** three times to make at least three commits as you work on your documentation improvements.

	Tasks	Git Commands
A.	Create a branch named <i>documentation</i> on your local repository.	git checkout -b documentation
B.	Switch to the <i>documentation</i> branch.	git checkout documentation
C.	Update your README.md file.	No git command needed
D.	Stage your changes.	git add README.md bikeshare.py
E.	Commit your work with a descriptive message.	bash git commit -m "doc: Add README and improve documentation"
F.	Push your commit to your remote repository branch.	git push origin documentation
G.	Switch back to the master branch.	git checkout main

3. Additional Changes to Documentation

In a real-world situation, you or other members of your team would likely be making other changes to the documentation on the documentation branch. To simulate this, follow the tasks below.

	Tasks	Git Commands
A.	Switch to the <i>documentation</i> branch.	git checkout documentation
B.	Make at least 2 additional changes to the documentation - this might be additional changes to the README or changes to the document strings and line comments of the bikeshare file.	No git command needed
C.	After each change, stage and commit your changes. When you commit your work, you should use a descriptive message of the changes made. Your changes should be small and aligned with your commit message.	bash git add . bash git commit -m "doc: Improve code comments"
D.	Push your changes to the remote repository branch.	git push origin documentation
E.	Switch back to the <i>master</i> branch.	git checkout main
F.	Check the local repository log to see how <i>all the branches</i> have changed.	git log --oneline --all
G.	Go to Github. Notice that you now have two branches available for your project, and when you change branches the README changes.	No git command needed

4. Refactor Code

Now you will be working in your local repository, on the Bikeshare code file to make improvements to its efficiency and readability. You should repeat steps C through E three times to make at least three commits as you refactor.

	Tasks	Git Commands
A.	Create a branch named <i>refactoring</i> on your local repository.	git checkout -b refactoring
B.	Switch to the <i>refactoring</i> branch.	git checkout refactoring
C.	Similar to the process you used in making the documentation changes, make 2 or more changes in refactoring your code.	No git command needed
D.	<i>For each change</i> , stage and commit your work with a descriptive message of the changes made.	bash git add bikeshare.py bash git commit -m "refactor: Improve code readability"
E.	Push your commits to your remote repository branch.	git push origin refactoring
F.	Switch back to the <i>master</i> branch.	git checkout main
G.	Check the local repository log to see how <i>all the branches</i> have changed.	git log --oneline --all
H.	Go to GitHub. Notice that you now have 3 branches. Notice how the files change as you move through the branches.	No git command needed

5. Merge Branches

	Tasks	Git Commands
A.	Switch to the <i>master</i> branch.	git checkout main
B.	Pull the changes you and your coworkers might have made in the passing days (in this case, you won't have any updates, but pulling changes is often the first thing you do each day).	git pull origin main
C.	Since your changes are all ready to go, merge all the branches into the master. Address any merge conflicts. If you split up your work among your branches correctly, you should have no merge conflicts.	git merge documentation git merge refactoring
D.	You should see a message that shows the changes to the files, insertions, and deletions.	No git command needed
E.	Push the repository to your remote repository.	git push origin main
F.	Go to GitHub. Notice that your master branch has all of the changes.	No git command needed

Submission:

This concludes the project.

- Please review this document to make sure you entered all the required response fields in all four sections.
- Download this document as a PDF file.
- Submit the PDF file on the Project Submission page within the Udacity Classroom.

Resubmission Note

Note to Reviewer:

For this resubmission, I updated the Git Commands Documentation to ensure all commit messages follow the Udacity Git Commit Message Style Guide (using prefixes such as chore:, doc:, and refactor:).

The previous reviewer indicated that it was acceptable to correct the report only without rewriting the Git history. Therefore, the commit messages shown in the PDF reflect the correct style guide, even if the original GitHub commits were created before I was aware of the required convention.